ABAP INTERVIEW PREPARATION

ABAP quick revision

Senior Cloud decisions first, followed by essential concepts and common traps

353 questions across 25 study segments. Reviewed 13 October 2026.

Use the short answers for oral rehearsal. The examples and pitfalls prepare you for follow-up questions. Choose the target SAP edition and release before applying an implementation detail.

Open abap-interview-studio.pages.dev online, or download the repository and open START_HERE.html for offline study. The PDF is a portable reading and printing edition.

Clean Core at a glance

Clean Core levels

Classify actual SAP dependencies and extension technologies

A	**Released SAP APIs and extension points**	No corresponding finding
B	**SAP-designated classic APIs and options**	Priority 3 / information
C	**Internal SAP objects with upgrade risk**	Priority 2 / warning
D	**Non-recommended objects or techniques**	Priority 1 / error

A-D describes Clean Core usage checks, not every ATC finding.

Verify the target edition, release, classification data and configured checks.

Use the correct scope

These levels describe extension dependency choices. The ATC mapping is for the configured Clean Core usage checks. Public edition and ABAP for Cloud Development do not become unrestricted simply because a classic option exists in another edition.

Senior ABAP Cloud architecture decisions

CLOUD-01 A new approval extension needs immediate document validation and an external supplier portal. Would you put everything on-stack or on BTP?

Keep the document check where it can reject the document before saving, using a supported local extension point. The supplier portal can be a separate BTP app if its users and release cycle justify that. Connect them through released remote interfaces. Make pending and failed steps visible. Choose based on timing, APIs, permissions, and recovery rather than putting everything in one place.

Remember Deploying side-by-side does not make synchronous cross-system validation atomic or automatically clean-core compliant.

CLOUD-02 A stakeholder says ABAP Cloud means moving the existing application to SAP BTP ABAP environment. How would you correct the proposal?

ABAP Cloud is a development model, not just a place to run code. It uses restricted language versions, released APIs, and models such as RAP on supported S/4HANA systems and SAP BTP ABAP environment. Moving to BTP is a separate decision. Check the data, business APIs, connectivity, and operational responsibilities available there before promising that an existing app will move unchanged.

Remember Cloud-compatible syntax does not establish identical application content, API availability, or deployment portability.

CLOUD-04 Two teams want independent releases but share pricing helpers and persistence types. How would you structure packages and software components?

Give shared code a clear owner and a small, stable interface. Packages organize code and access; software components also affect delivery and lifecycle dependencies. Keep private database types with their owning component. Avoid circular dependencies and define delivery order. Supported component relations can allow targeted access, but access permission alone does not promise that an interface will stay compatible.

Remember More software components do not create independent lifecycles when their dependencies still require lockstep delivery.

CLOUD-18 What would you require before approving a senior team's ABAP Cloud architecture for production?

Require evidence that the main business rules, permissions, transaction boundaries, and recovery paths work on the intended runtime. Check released dependencies, realistic workload, monitoring, retention, and deployment order. Give important assumptions an owner and a test result. Distinguish a prototype from a production-tested solution. Remaining risk can be accepted deliberately, with practical controls and clear responsibility.

Remember A clean syntax check, green ATC run, or attractive diagram is not an end-to-end production-readiness certificate.

ABAP Cloud: senior RAP transactions and behavior design

SRAP-01 A new approval app has custom persistence, while its posting step must reuse an existing transactional API. Which RAP implementation type would you choose?

Start with managed RAP for the new approval tables. If an eligible API must perform the actual writes while RAP can manage the pending changes, consider managed RAP with unmanaged save. Use fully unmanaged RAP when existing logic must also handle interaction. Check the API's locks, permissions, numbering, buffering, and commits. Calling it a BAPI does not prove it fits.

Remember Choosing unmanaged merely because the application is complex, or assuming managed with unmanaged save removes RAP transaction rules.

SRAP-03 A developer puts ordinary business validation in SAVE because the final data is available there. What would you change?

Move predictable business checks to the supported early save checks, after needed calculations and before late saving. Late save normally writes data that has already passed those checks. An error message is not a general rollback mechanism there. SAP does document special failure-capable saver options for some BAPI integrations. Verify that exact option and release instead of assuming all late failures work alike.

Remember Saying either 'all errors can be returned from SAVE' or 'no supported late-save failure mechanism exists anywhere'.

SRAP-04 One action updates two local RAP BOs and then calls a remote payment service. Can the whole action be atomic?

Compatible local RAP objects can take part in the same RAP-controlled SAP transaction. A normal HTTP payment call cannot join that database transaction. If payment succeeds and local saving fails, local rollback will not undo payment. Model pending work and reliable delivery, or an explicit compensating process. Define timeout, retry, and reconciliation behavior instead of promising one atomic action.

Remember Assuming that calling HTTP inside an action enrolls the remote system in the SAP database transaction.

SRAP-08 A posting API returns the final document number only when called. How would you integrate numbering without publishing incorrect keys?

Keep preliminary identities until the supported save process assigns final document keys. Map %pre to the final %key during number adjustment. SAP documents a BAPI pattern that calls creation in ADJUST_NUMBERS with a suitable failure-capable saver. Check its availability, API release status, and error rules on the target system. Do not present an earlier identifier as a committed document number.

Remember Do not use SELECT MAX plus one, lose the preliminary-to-final key mapping, or assume ordinary ADJUST_NUMBERS supports recoverable failures without the documented special saver.

SRAP-12 A request changes CompanyCode to an unauthorized value and also makes its dates inconsistent. Would you use prechecks or validations?

Use a supported precheck to reject a disallowed incoming company-code change before it enters the buffer. Use the appropriate validation to reject inconsistent final dates. Keep the complete RAP authorization design as well; neither mechanism replaces it. For partial updates, combine supplied fields with the existing values needed for the check. Do not assume the request contains every field.

Remember Treating prechecks as a global security bypass or believing a UI mandatory marker enforces all backend invariants.

SRAP-16 An older RAP BO works without strict mode. The team wants to release it for cloud consumption by adding strict(2). Is that sufficient?

No. strict(2) adds modeling and implementation checks; it does not release the object as an API. Review base and projection behavior together, including locks, permissions, ETags, and composition relationships. Check derived-type changes and eligible dependencies. A released API also needs a stable supported interface and lifecycle plan. Syntax success alone proves neither compatibility nor secure operation.

Remember Treating strict(2) as a clean-core grade or API-release switch, or enabling it only on one layer and ignoring resulting contract changes.

Senior CDS, query contracts and security

MODEL-01 A new consumption entity exposes orders outside the user's sales organization. The base CDS has DCL. What do you investigate?

Trace the entity that the service actually reads and its access rules. DCL on the base view does not automatically protect every higher view or navigation path. Define the required restrictions, using supported inheritance and mappings where appropriate. Test both allowed and denied data. A UI filter or #CHECK annotation alone cannot create the missing security rule.

Remember Do not add #CHECK and assume it creates a missing rule.

MODEL-02 An approver may update organization A. Can an update move an order from A to B using only existing-instance authorization?

Permission to update an order in A does not automatically allow moving it into B. Check both the current object and the proposed target values at the appropriate operation boundary. Supported prechecks can help reject disallowed input before it enters the buffer. Cover creates, updates, and derived changes. Keep the complete authorization and business validation design, not just the existing-instance check.

Remember Do not equate authorization for the current object with authorization for any future object state.

MODEL-03 A custom query returns only authorized rows but an unrestricted total count. Is it secure?

No. The count must describe the same authorized, filtered result as the returned rows, before paging. A custom provider cannot assume remote or manually assembled data is protected automatically by database access control. Apply the same restrictions to data, count, and supported aggregates. If an operation cannot be answered securely and correctly, reject it clearly instead of returning misleading information.

Remember Filtering only the displayed page leaves count, aggregates and existence checks exposed.

MODEL-04 A query-provider filter cannot be converted to simple ranges. A developer catches the exception and returns everything. What is the correction?

Do not turn a filter-conversion failure into an unrestricted read. Preserve the original AND/OR grouping using a supported expression representation or another safe translation. If the provider cannot support the filter, reject it clearly. Keep mandatory authorization restrictions separate from user filters so neither disappears on an error. Handle dynamic field names and values through supported safe APIs.

Remember A swallowed filter exception must not become an unrestricted selection.

MODEL-14 A CDS unit test passes, but the same entity leaks data for a restricted user. What may the test be missing?

Check whether the test actually evaluates access control. CDS doubles used to test joins may isolate or disable DCL, so a passing query test is not automatically a security test. Use supported access-control test setup with meaningful permission data where available. Add real service tests with restricted identities. Keep query correctness and deployed authorization evidence separate.

Remember A green database-double test does not prove deployed authorization.

Senior RAP testing, performance and production

OPS-01 How would you test a Submit action without replacing the business object you are supposed to verify?

I run the real Submit action and keep the business rules I want to test. I replace only outside dependencies, such as another service or business object, with suitable test doubles. Then I call the action, often through EML, and check its results, errors and saved state. Mocking Submit itself would not test Submit.

Remember Do not label a test of a mocked Submit response as a test of Submit.

OPS-05 How do you prove optimistic concurrency rather than merely calling the same update twice?

I use two independent clients that read the same version before either saves. The first update should succeed; the second should be rejected when it sends the old ETag. I check that the first user's data remains intact. Draft locks and the draft total ETag need separate tests because they protect different parts of the editing flow.

Remember Two updates in one shared transaction are not a realistic two-user concurrency test.

OPS-11 A remote approval succeeded, but the RAP caller timed out. How should an operator recover?

A timeout leaves the result unknown until I check it. I use the request's stable reference to find the remote approval before submitting again. If it exists, recovery should reuse that result. If it cannot be determined, the case needs a visible status and an owner. Local rollback cannot erase an approval already committed remotely.

Remember A timeout is not evidence that the other system performed no work.

OPS-14 What belongs in a RAP release gate beyond a passing ATC run?

ATC is one check, not proof that the feature works. I also test the business rules, failures, permissions and integrations affected by the change. Changes involving saves or concurrency need matching rollback and conflict tests. I record the version and environment for each result, and clearly mark anything not run rather than count it as passed.

Remember Choose tests for the failure risks this change creates; a generic checklist alone is not enough.

Senior Clean Core governance and tradeoffs

GOV-01 A project claims Level A because its frontend uses RAP. How would you assess the extension?

Check the whole extension, including wrappers and the SAP objects they call. RAP and Fiori do not decide the Clean Core level. A depends on released contracts; B uses classified classic options; C uses internals; D uses non-recommended dependencies or techniques. Record the actual classification for each path on the target system. Do not call a mixed solution entirely Level A.

Remember RAP, BTP deployment, a customer namespace, or a green UI test is not proof that every underlying dependency is Level A.

GOV-02 An API is released with C1. A team assumes that means Clean Core Level C and permission to expose it remotely. How do you respond?

These are two different naming systems. C0 means Extend, C1 means Use System-Internally, and C2 means Use as Remote API. They are release-contract purposes, not Clean Core levels A to D. Check the object's visibility for the intended caller too. A C1 release does not by itself authorize remote exposure or classify the dependency as Level C.

Remember C0/C1/C2 and Clean Core A/B/C/D are different classifications; release for one purpose does not release every possible usage.

GOV-04 An urgent release needs an ATC exemption for an internal dependency. What makes the exception governable?

Make the exception specific: business need, exact finding, source scope, target edition, alternatives, risk owner, and supporting tests. Give it an expiry or review trigger and a realistic fix plan. An ATC exemption changes how a finding is handled. It does not make forbidden code legal or turn the dependency into Level A. The release decision must accept the remaining risk explicitly.

Remember An approved exemption is not a reclassification to Level A and must not become a general permission to bypass ABAP Cloud restrictions.

GOV-10 The direct dependency scan is clean, but an upgrade still breaks the extension. What could the governance model be missing?

Check what the scan did not cover: indirect calls, dynamic selection, configured implementations, remote interfaces, and data assumptions. Verify its package scope, check version, and classification data too. Connect static findings with a dependency map and process tests. Clean direct references are useful evidence, but they do not prove that every runtime path will survive an upgrade.

Remember A green direct-reference check is not proof that the whole solution has no unsupported or semantically fragile dependencies.

GOV-18 An interview panel shows an A-D chart with ATC priorities. What would you clarify before treating it as a release gate?

Explain that the A-D priority mapping belongs to particular Clean Core checks. Ask which variant, checks, classification data, object scope, and target release produced the results. Other ATC priorities can describe security or performance issues instead. Define what the release gate proves, which exemptions remain, and which runtime tests are still needed. Save enough run information to repeat the assessment.

Remember Do not interpret every priority 1 finding as Level D, every priority 2 as Level C, or absence of findings as proof of complete Level A coverage.

ABAP foundations

ABAP-01 How do you choose between standard, sorted, and hashed internal tables?

I choose the table by how I will read it. A standard table is useful for processing rows in sequence or by position. A sorted table keeps rows in key order and helps with key ranges. A hashed table is useful for finding one row by its complete unique key. I also consider how often rows change and how much memory the table needs.

Remember A hashed primary key is unique; that does not mean every field value must be unique or that arbitrary partial-key searches are constant time.

ABAP-07 How do you explain an SAP LUW versus a database LUW?

A database LUW is one database transaction, ending with commit or rollback. An SAP LUW is the whole business change that the application treats as one unit. In classic dialog processing, it can span several database LUWs. SAP locks and the save process help keep that business change consistent. The application or framework that owns the transaction decides when to save.

Remember A database commit caused by a work-process boundary is not permission to split a business operation into independently committed fragments.

ABAP-08 Who should issue COMMIT WORK, and what does AND WAIT guarantee?

The caller responsible for the complete business operation should commit. A helper must not save half the work unexpectedly. In classic update processing, COMMIT WORK AND WAIT waits for high-priority updates: sy-subrc is 0 for success and 4 for failure. Without AND WAIT, sy-subrc is 0 without confirming the asynchronous update result. Neither form waits for every external follow-up.

Remember COMMIT WORK AND WAIT is not a distributed transaction guarantee and does not authorize commits inside RAP handlers.

OO design and Clean ABAP

OO-02 What can an ABAP interface contain, and when would you use one?

An ABAP interface describes what an implementing class provides. It can contain methods, types, constants, attributes and events, not just method names. I use a small interface when callers need one clear capability and different implementations can provide it. This also lets a test supply a controlled replacement. Specialized interfaces, such as BAdI interfaces, can have additional rules.

Remember An interface is not an abstract class, and a collection of similarly named methods alone does not establish a meaningful common contract.

OO-03 How do ABSTRACT and FINAL affect classes and methods?

An abstract class cannot be created directly, but it may contain working shared code. It does not have to contain an abstract method. An abstract method leaves implementation to a subclass before that subclass can be concrete. FINAL on a class prevents subclasses. FINAL on a method prevents further redefinition of that method.

Remember Do not import another language's virtual keyword or method-overloading rules into ABAP; use ABAP's actual redefinition and signature rules.

OO-07 How do dependency injection and dependency inversion differ?

Dependency inversion means business code depends on the capability it needs, usually an interface, rather than one technical implementation. Dependency injection means someone else supplies that implementation, often through the constructor. The first is a design rule; the second is a way to connect objects. Together they make dependencies visible and easier to replace in tests.

Remember Injecting a concrete class everywhere may aid construction while still leaving the business layer tightly coupled to infrastructure.

ABAP SQL, HANA, and performance

SQL-02 What is the empty-table trap in FOR ALL ENTRIES?

An empty FOR ALL ENTRIES driver is dangerous: the WHERE condition is ignored, so the query can read all rows in scope. I check that the driver is not empty before selecting. I also decide how duplicates should be handled. FOR ALL ENTRIES removes duplicate result rows, which can matter when selected columns do not uniquely identify records.

Remember An empty FAE driver does not mean no rows; a separate filter in the same WHERE clause is not a safe replacement for the guard.

SQL-04 How can a LEFT OUTER JOIN accidentally behave like an inner join?

A LEFT OUTER JOIN keeps left-side rows even when no right-side row matches. The missing right side contains SQL NULL values. A WHERE condition demanding a right-side value can remove those rows again. I put a condition in ON when it defines which right rows may match. I use WHERE when I really want to filter the final result.

Remember Moving conditions between ON and WHERE is not always semantics-preserving for outer joins.

SQL-10 Which tools help locate ABAP, database, and memory performance problems?

Start with the slow business request and locate where its time or memory goes. Where available, ST03N/STAD help identify costly workloads or requests, SAT traces ABAP execution, and ST05 traces SQL. ST12 combines relevant traces. SQL Monitor helps prioritize SQL across a wider workload; a database plan explains one statement. Memory analysis needs its own evidence.

Remember The slowest individual SQL statement may not be the largest total cost if another cheap statement is executed thousands of times.

CDS data modeling

CDS-03 Association or join and how do you select cardinality?

A join combines sources in the current query. An association defines a relationship that a consumer can follow, often producing a join when used. Cardinality describes how many related rows can really match the ON condition. It must match the data. Declaring a relationship to-one when several rows can match can lead to incorrect results.

Remember Associations are not automatically faster than joins and do not enforce referential integrity.

CDS-07 Does DCL on a base view automatically protect every higher view?

No. Access control on a lower CDS view does not automatically protect every view built on it. Design the read rules for the entity that callers actually access. Supported DCL inheritance can reuse conditions with the required mappings. The #CHECK annotation alone creates no rule. RAP permissions for changing objects are a separate check.

Remember DCL on one layer does not automatically secure an arbitrary view-on-view stack.

CDS-11 How do CDS keys and database keys relate?

A CDS key tells consumers which fields identify a row. It does not force the query result to be unique. A database key enforces uniqueness in a table, but joins and grouping can change the result. RAP and OData need a stable, truly unique identity. Check the complete result, not just the key declarations.

Remember Marking an element key does not remove duplicate rows.

RAP foundations and transactions

RAP-02 How do you choose managed versus unmanaged RAP?

Use managed RAP when the framework can handle your object's changes and database writes. Use unmanaged RAP when your implementation must handle those jobs, often because an existing business API already owns them. Managed RAP with unmanaged save is a middle option: RAP handles changes, and your code handles persistence. In every option, RAP still controls the transaction.

Remember Unmanaged does not permit arbitrary COMMIT WORK or database changes anywhere in a handler.

RAP-04 Where may COMMIT ENTITIES be called?

Call COMMIT ENTITIES from an external ABAP program that owns an EML transaction. Inside a RAP handler or saver, let the framework finish the transaction. Do not call COMMIT ENTITIES or COMMIT WORK there. This rule also applies to actions, validations, and determinations. The OData request framework owns the save boundary for its requests.

Remember Moving the commit from validation into SAVE or CANCEL does not make it valid.

RAP-07 How do FAILED REPORTED and MAPPED differ?

FAILED says which instance or operation was rejected. REPORTED carries messages, including their severity and the field or instance they concern. MAPPED links a caller's creation identifier to the created object's identity where the operation supports it. These structures do different jobs. An error message in REPORTED alone does not necessarily make an operation fail.

Remember Error severity in REPORTED is not a substitute for FAILED; determinations do not expose FAILED.

RAP-10 What do %cid %cid_ref %tky and %pid mean?

These fields identify objects at different stages of a RAP request. %cid labels a new instance, and %cid_ref refers to that creation, for example when adding its items. %tky is the transactional key, including relevant context such as draft state. %pid is a preliminary identifier for late numbering. None of these names means they are interchangeable.

Remember %tky means transactional key, not temporary key; %cid is not a persisted business document number.

RAP-11 Early numbering versus late numbering?

Early numbering gives an instance its final key during interaction. Late numbering keeps a preliminary identity and assigns the final key late in saving. A technical UUID and a business document number can be separate fields with different rules. Late numbering can avoid consuming numbers for some rejected changes. It does not, by itself, promise a gap-free business sequence.

Remember Late numbering alone does not guarantee a gap-free sequence, and SELECT MAX plus one is unsafe under concurrency.

RAP-12 What problem does draft solve?

Draft saves unfinished editing work separately from active business data. It lets a user continue across HTTP requests or return after an interruption. RAP provides operations such as Edit, Resume, Activate, and Discard; Prepare can run selected checks and calculations. Activation moves valid draft changes into active processing. The request still needs its save sequence to finish.

Remember Every BO is not automatically draft-enabled, and a custom Submit action is not automatically draft activation.

RAP service design and advanced behavior

ADV-01 How do pessimistic locks and ETags complement each other?

A lock stops conflicting processing while it is held. An ETag lets a write request check whether the data has changed since the client read it. They solve different problems and are often needed together. RAP declares the relevant lock and version handling in behavior. Draft scenarios also need total-ETag handling for relevant changes to active data.

Remember Locking and logging are different concepts; an ETag is not a log and does not reserve the object.

ADV-02 What is the difference between global and instance authorization in RAP, and where do DCL and prechecks fit?

Global authorization asks: may this user perform Update in general? Instance authorization asks: may this user update this particular record? Here, instance means a business-object record, not an ABAP class instance. DCL controls which rows a read returns. A precheck can reject incoming values before they enter the transactional buffer.

Remember Global authorization, instance authorization, DCL, prechecks and feature control solve different problems. A global pass or disabled button does not secure every record or incoming value.

ADV-06 What is augmentation and how does it differ from additional save?

Augmentation changes or adds supported input at a RAP projection boundary before the underlying business object handles it. It helps adapt a particular consumer's input. Additional save performs extra persistence-related work in a suitable managed save scenario. These mechanisms run for different reasons. Neither is a general hook for arbitrary work before every save.

Remember Augmentation is not a generic before-save hook for copying every virtual element into a table.

ADV-13 How do static and dynamic feature controls differ?

Static restrictions are declared in the behavior definition, such as a readonly field. Dynamic feature controls decide availability from current context, such as making approved requests uneditable. They describe business availability, not who has permission. Check the controls supported for the operation and release. For one operation, choose global or instance feature control; do not declare both.

Remember A dynamic UI state does not prove security, and mandatory metadata does not replace all backend business validations.

OData Gateway and Fiori integration

ODATA-01 OData V2 vs V4: what really changes, and what are the benefits of each?

Both let an app read and change business data over HTTP. V4 gives a clearer API contract: separate actions and functions, a date-only type, and more expressive queries. V2 is valuable when existing SAP apps or partners already depend on it. For a new RAP app, I normally choose V4 when the backend and UI support the required features. V4 is not an automatic speed upgrade, and a V2 client cannot simply switch its URL and become a V4 client.

Remember Do not claim that V2 only supports XML, V4 introduced $batch, or only V4 supports RAP/draft. Both can use JSON. Protocol support, SAP provider support and UI5 model capabilities are separate checks.

ODATA-04 How do paging and total count work in V2 and V4?

The total count tells the client how many rows match the filter, not how many fit on one page. Filter first, then count and page with a stable order. V2 commonly uses $inlinecount=allpages; V4 uses $count=true for an inline count. When the server returns a next link, the client should follow it.

Remember Use the response API of the actual provider. Do not assume every Gateway implementation has one generic SET_COUNT method.

ODATA-07 Is a batch request one database transaction?

A batch is an envelope containing several requests. It does not automatically make them one transaction. Changesets provide atomic groups in multipart batches; OData 4.01 JSON batch has atomicity groups. I check what the service supports and inspect every inner result. An outer success response does not mean every operation succeeded.

Remember HTTP 200 for a batch does not mean every operation inside it succeeded.

Clean Core levels and extensibility

CORE-02 What are Clean Core levels A B C and D?

The levels describe which SAP dependencies and extension techniques custom code uses. A uses released APIs and extension points under their supported contracts. B uses SAP-designated classic APIs and supported classic options. C uses SAP internals that may change. D uses explicitly non-recommended objects or techniques. Check the actual classification for the product and use involved.

Remember A familiar API is not automatically B, and every unreleased object is not automatically D.

CORE-03 How do the levels map to ATC messages?

For the configured Clean Core usage checks, A has no corresponding finding. B gives priority 3 information, C gives a priority 2 warning, and D gives a priority 1 error. This mapping applies to those checks. Other ATC checks may use priorities for different problems. A green run is useful only when the right checks and classification data actually ran.

Remember No ATC findings does not prove that the entire application is secure, correct or fully Level A.

CORE-08 Does wrapping an unreleased SAP API make it Level A?

No. A released custom wrapper can give its callers a controlled interface where the platform allows that design. The wrapper still calls the underlying SAP object, with that object's classification and upgrade risk. Review the full call chain. Releasing your own interface does not make an internal or non-recommended SAP dependency become Level A.

Remember A wrapper is not a mechanism for laundering an internal or prohibited dependency into an entirely Level A solution.

CORE-10 How do you choose key-user on-stack and side-by-side extensibility?

Choose the simplest supported option that meets the business need. Key-user tools fit supported field and process adaptations. On-stack developer extensibility fits logic that needs close access to local business data or the transaction. Side-by-side fits a more independent app or cross-system process. Use supported remote interfaces and plan how separate systems recover when one fails.

Remember These are placement and development choices, not automatic A-D grades or fixed implementation times.

CORE-15 What if no released API covers the business requirement?

First recheck standard features, configuration, and supported extension points on the target release. Consider a different process or placement if it meets the requirement. Where classic development is allowed, assess designated classic APIs and any remaining internal dependency under the exception policy. Request a supported API when needed. Do not assume a classic workaround is possible in public edition.

Remember Do not immediately choose a modification, and do not assume public edition permits a classic workaround.

BTP and cloud integration for ABAP developers

BTP-04 What are a service instance binding and service key?

A service instance is a service resource you have created. A binding connects an application to that resource and gives it the required connection information. A service key supplies credentials for a supported external-client use case. These are different objects; creating a binding does not always create a separate service key.

Remember default-env.json is a local development convention, not the universal Cloud Foundry credential transport.

BTP-07 How do remote transactions differ from local RAP transactions?

A remote API usually saves data in a separate transaction. If my local RAP transaction rolls back, the remote system may still keep its change. I therefore plan how to find the original result, retry safely and handle partial success. A timeout means the outcome may be unknown; it does not prove the remote operation failed.

Remember Calling HTTP inside an action does not make the remote change part of the local RAP database transaction.

RICEFW, enhancements, and workflow

RICEFW-11 How should a RAP submit action start an approval process reliably?

Submit should check the request and change its state through RAP's transaction rules. The external approval process should start in a way coordinated with a successful save. I use the supported event or integration mechanism for that release. Stable identifiers and duplicate handling stop retries from starting several workflows for the same submission.

Remember Calling an external workflow endpoint before save succeeds can start approval for a document that never committed.

RICEFW-15 Which S/4HANA changes can affect existing ABAP developments?

I check the simplification information and custom-code findings for the target S/4HANA release. Important areas can include MATDOC, ACDOCA, Business Partner and changed output or transactions. I trace how the existing application uses the affected objects. Replacing a table name mechanically is not enough; the business meaning and supported access may have changed.

Remember Replacing a legacy table name with MATDOC or ACDOCA mechanically is not a migration strategy.

Testing security and delivery

TEST-03 How would you test a RAP business object?

I test the real RAP business object with controlled data, using suitable EML tests where supported. I check successful changes, rejected changes, messages and what is saved. I also cover authorization and conflicting edits at the appropriate test level. Service and UI tests cover different behavior. Available RAP test frameworks depend on the target release.

Remember A mock Fiori success or a doubled BO does not establish that the real RAP implementation works.

TEST-06 What security tests would you include?

I test what a user may do and what they must be prevented from doing. That includes direct API reads and actions, not only visible buttons. I check realistic roles, company or organizational limits, bulk requests and sensitive fields. Login, CDS access control and operation authorization are separate checks. Error messages must also avoid revealing data the user may not see.

Remember Feature control, UI.hidden and disabled buttons are not sufficient security boundaries.

HTTP, REST, JSON and XML

HTTP-01 What is the difference between HTTP, REST, OData and a payload format?

HTTP defines how requests and responses work. REST is a set of design rules for an API. OData adds rules for data services, such as querying related records. JSON and XML are formats for the data you send. Using JSON over HTTP does not, by itself, make an API RESTful.

Remember Do not use REST, HTTP and JSON as interchangeable names.

HTTP-04 What do safe and idempotent mean, and how do they affect retries?

Safe means the requested operation is read-oriented. Idempotent means repeating the same request has the same intended effect as doing it once. PUT and DELETE are idempotent by HTTP semantics, although their responses may differ. POST and PATCH need a specific guarantee before I repeat a business change after a timeout.

Remember Do not blindly retry a potentially committed business creation because the response was lost.

HTTP-07 What must you preserve when mapping JSON to ABAP?

I preserve field names, data types and the meaning of missing values. JSON distinguishes an absent property, null and an empty string; an ABAP update may need to distinguish them too. Arrays keep their order, but object-property order is not a reliable contract. I also protect number precision and identifiers with leading zeros.

Remember Treating missing and null fields as the same initial ABAP value can accidentally change or clear data.

HTTP-08 What XML concepts matter in an integration interview?

XML represents data with elements, attributes and nested content. Well-formed XML follows syntax rules; schema validation checks further rules about its structure and values. Namespace prefixes are aliases, so I identify names using the namespace URI and local name. I use a real XML parser rather than splitting the text by hand.

Remember String splitting is not an XML parser. Correct XML syntax also does not prove valid business data.

Function modules, RFC, BAPI, parallel processing, and jobs

RFC-02 How do synchronous, asynchronous, transactional, queued, and background RFC differ?

sRFC waits for a reply; aRFC lets the caller continue and collect results later. tRFC records a unit for processing after commit, with durable retry. qRFC adds ordering through queues, while bgRFC supports transactional and queued units with newer infrastructure. I choose by response, retry and ordering needs, within the target system's supported options.

Remember STARTING NEW TASK does not provide durable message retry. Separate queues do not automatically have one shared processing order.

RFC-04 When should BAPI_TRANSACTION_COMMIT or BAPI_TRANSACTION_ROLLBACK be called, and what does WAIT mean?

The code that owns the BAPI transaction decides when to commit or roll back after checking the business results. For a remote BAPI, transaction control must reach the correct remote context. WAIT requests completion of high-priority updates; it does not wait for every later workflow or interface. I also check the specific BAPI's documented exceptions.

Remember Check the BAPI's actual success rules and return messages. Local COMMIT WORK cannot commit an unrelated remote BAPI context.

RFC-07 How do you prevent parallel RFC processing from exhausting resources?

I limit how many tasks can run at once and give each task a useful-sized package. In classic ABAP, DESTINATION IN GROUP can use a configured RFC server group. Resource availability can change, so I handle RESOURCE_FAILURE and reduce or delay dispatch. I measure the result: more parallel tasks can increase waiting and make the job slower.

Remember A resource count is a snapshot, not a reservation; hard-coded maximum concurrency can overload the system or increase lock contention.

RFC-10 How do you schedule and troubleshoot a classic background job?

In a classic system, I schedule job steps, variants, user and start conditions, commonly in SM36. In SM37, I inspect status, job log, steps and spool. I then follow application errors or dumps. Before restarting a canceled job, I check what it already saved; cancellation does not undo earlier commits.

Remember SM37 Finished is not proof that every business record succeeded, and canceling a job does not reverse its earlier commits.

IDoc, ALE, AIF, and interface operations

INTF-01 What does an IDoc contain, and how do message type, basic type, and extension differ?

An IDoc has control data, business data in segments and a processing-status history. The message type says what business message it represents. The basic type defines the standard structure, and an extension adds supported customer segments. Sender and receiver must agree on these parts; adding a segment definition does not automatically fill or process its values.

Remember Message type and basic type have different jobs. An IDoc number is also not automatically the target business document number.

INTF-07 What is SAP Application Interface Framework, and what is it not?

AIF helps teams process, monitor and recover interface messages with useful business context. Depending on configuration, it can provide mappings, checks and actions. It does not replace the underlying transport or automatically cover every interface. I verify the target product's supported features and usage rights before proposing a custom AIF solution.

Remember AIF does not replace the transport. Its presence does not prove unrestricted custom-use rights or the same features in every product.

INTF-11 When would you use OData, IDoc, or AIF?

OData provides HTTP-based queries and business operations. IDoc provides structured messages for established application-integration scenarios. AIF adds processing, monitoring and recovery around supported interfaces. They can work together because they solve different needs. I choose the available business contract and delivery behavior first, then decide how operators will monitor it.

Remember AIF is not a third wire protocol, OData is not limited to UI use, and IDoc is not restricted to batch-only dispatch.

Practical ABAP: values, expressions, and failure paths

BASE-01 What changes when a method receives a CHANGING parameter by value instead of by reference?

By reference, the method works on the caller's data directly. With VALUE on a CHANGING parameter, it works on a local value and copies it back when the method finishes normally. If an exception leaves the method, that normal copy-back does not happen. This can protect the caller's value, but it does not roll back database writes or changes to shared objects.

Remember Pass-by-value is not a transaction rollback and does not clone objects behind reference parameters.

BASE-04 A dynamic ASSIGN fails, but IS ASSIGNED is still true. How can the program update the wrong field?

The field symbol may still point to the previous successful target after a failed dynamic ASSIGN. IS ASSIGNED only says it points somewhere; it does not prove the latest lookup worked. I check the ASSIGN result immediately. Where supported, ELSE UNASSIGN makes failure clear. Otherwise I unassign before the lookup and handle failure before writing through the field symbol.

Remember An assigned field symbol is not evidence that the most recent dynamic ASSIGN succeeded.

BASE-06 An import must reject precision loss rather than silently round or truncate. Where does EXACT help?

EXACT checks that an assignment or calculation can produce its target value without losing information. It raises an appropriate exception when that is impossible. CONV uses normal conversion rules and is not the same check. I catch conversion failures at the input boundary and explain them to the caller. I still validate business rules, because an exact value can still be forbidden.

Remember EXACT does not prove that a technically representable value is an allowed business value.

BASE-12 A report accepts a filter value and a sort-column name. Is escaping quotes sufficient for safe SQL?

No. A filter value and a column name are different kinds of input. I pass values through host variables in static ABAP SQL where possible. For a configurable column or dynamic clause, I map approved choices to known SQL elements and reject anything else. Escaping quotes in a value does not make arbitrary SQL text safe or authorized.

Remember Input that is safely quoted as a literal is not therefore safe as SQL syntax or an authorized data access.

BASE-16 An authorization helper returns an empty range table. Why is WHERE field IN range unsafe as the only access restriction?

An empty ABAP selection range means no restriction for IN. It does not mean the user is allowed no values. If an authorization lookup returns no allowed plants, I return no data or reject the request before querying. I keep mandatory access restrictions separate from optional user filters. Nonempty ranges also have inclusion and exclusion rules, not just list membership.

Remember An empty ABAP selection range is not the same as an SQL list containing zero allowed values.

Practical OO: contracts, state, and substitutability

OODEEP-01 A method named reserve returns a reservation ID. Which promises must be explicit beyond its parameter types?

I explain what inputs are accepted, what success means and what can change before a failure. The caller also needs to know whether the reservation is already saved, whether repeating the call is safe and who commits. Parameter types cannot answer those questions. I document these rules and test them so another implementation cannot quietly change the caller's expectations.

Remember RETURNING an identifier does not itself promise committed persistence, and a method name cannot replace a transaction contract.

OODEEP-09 A repository raises a low-level exception. Should a business service catch CX_ROOT and return an empty result?

Usually no. Returning an empty result can make a technical failure look like a valid not-found result. I catch the failure where I can recover or turn it into a meaningful service error. If I wrap the exception, I keep PREVIOUS so the original cause remains available. Missing data, a rejected business request and an unavailable dependency should remain distinguishable.

Remember Broad catch-and-default handling is not resilience when it changes an unknown outcome into valid-looking data.

Practical ABAP design patterns and trade-offs

PAT-01 Approval rules vary by country, while the application workflow should remain stable. Strategy or Template Method?

I would use Strategy for country-specific approval rules. The coordinator keeps the workflow, and the selected strategy returns the rule decision. Template Method fits a shared sequence where subclasses intentionally fill in a few fixed steps. Here the rule can vary without changing the coordinator class. I would only choose inheritance if the shared sequence and permitted overrides are clear requirements.

Remember Choosing inheritance solely to share code, or allowing a rule implementation to take over transaction ownership.

PAT-03 A legacy posting API has awkward parameters and a three-step prepare/check/post sequence. Would you build an Adapter, a Facade, or both?

An Adapter can translate the application request and errors into the legacy API format. A Facade can offer one posting operation over prepare, check and post. Both may be useful, but they must preserve failures, authorization and commit rules. I inspect the actual underlying calls. Neither wrapper makes an unreleased API available to ABAP Cloud or removes an internal commit.

Remember Treating any wrapper as both a supported API and a transaction adapter without inspecting its dependencies.

PAT-08 A repository interface improves testability, but its RAP implementation reads managed persistence directly. What can go wrong?

A testable interface can still hide an incorrect implementation. Direct table reads can miss changes waiting in the RAP transaction buffer. I first define whether the repository should read saved data or the current transaction state. For BO logic that needs current changes, I use suitable EML access. I do not write managed tables directly and bypass the RAP behavior checks.

Remember Equating a mockable interface with correct behavior or recreating RAP's transactional buffer in a second general-purpose cache.

PAT-09 A singleton caches the current approver and makes tests pass alone but fail as a suite. How do you redesign its lifetime?

I would remove the current approver from long-lived shared mutable state. I would pass the relevant context explicitly and use an operation-scoped service, or a cache with clear keys and refresh rules. An ABAP singleton normally belongs to an internal session, not the whole system. Its lifetime can still span several operations or tests. Forcing test order does not fix that design.

Remember Claiming singleton means one system-wide instance or solving hidden state solely by forcing test order.

Supporting skills: HANA decisions and source-to-release delivery

SQLX-01 A CDS table function joins client-dependent purchasing data in SQLScript, writes a helper table, and assumes the outer CDS DCL protects every internal read. Would you approve it?

No. A CDS table function's AMDP database function must be read-only under its documented rules. I remove the write and check every source and join for correct client handling. Business authorization is a separate check: an outer CDS rule does not automatically protect every internal SQLScript read. I also verify target support and allowed dependencies.

Remember The CDS table function is read-only. ABAP Cloud also restricts AMDP procedures to reads; Standard ABAP procedure capabilities are broader. None of these options grants permission to read any SAP table.

DELIV-01 A pull request passes local linting and its ABAP sources are pushed with abapGit. The team calls the cloud feature deployed. What evidence and delivery steps are missing?

Git stores source history, and abapGit transfers supported development objects. Neither proves that the feature is deployed and usable. I choose the delivery route supported by the actual ABAP product, then verify the target version, activation, tests and configuration. A real service check must show that the intended user can work and a denied user cannot.

Remember An abapGit push is not a transport release, and gCTS is not another name for abapGit. Delivery routes differ across ABAP Cloud products.